SOLAR▸SWARM

API Reference
Martin Pham | Perth, AU

API Endpoint Reference

The system exposes four Flask services on separate ports. All use JSON request/response bodies unless noted. Security headers (`X-Content-Type-Options`, `X-Frame-Options`, `Cache-Control: no-store`) are attached to every response.

Port Map

PORT	SERVICE	FILE
5000	Human Gate API	`api/human_gate.py`
5001	GHL Webhook Server	`webhooks/ghl_handler.py`
5002	Voice Webhook Server	`voice/call_handler.py`
5003	Dashboard API	`api/dashboard_api.py`

Human Gate API — Port 5000

Authentication: All endpoints except `/health` require:

```
Authorization: Bearer <GATE_API_KEY>
```

Set `GATE_API_KEY` in `.env`. If not set, auth is bypassed with a warning (acceptable for local dev; always set in production).

Rate limit: 20 requests per minute per IP on approve/reject endpoints.

GET /health

Public — no auth required.

Response:

```
{
  "status": "ok",
  "service": "human-gate",
  "circuit_breaker": "green",
  "halted": false,
  "timestamp": "2026-03-09T08:00:00"
}
```

```
curl http://localhost:5000/health
```

GET /pending

List all experiments awaiting human review (status = 'pending').

Response:

```
{
  "count": 2,
  "experiments": [
    {
      "id": 42,
      "idea_text": "...",
      "confidence_score": 7.2,
      "devil_score": 4.1,
      "vertical": "solar_australia",
      "bucket": "explore",
      "created_at": "2026-03-09T07:00:00"
    }
  ]
}
```

```
curl -H "Authorization: Bearer $GATE_API_KEY" http://localhost:5000/pending
```

POST /approve/

Approve an experiment and allocate budget using Kelly Criterion.

Request body (optional):

```
{
  "approved_by": "martin"
}
```

Response 200:

```
{
  "status": "approved",
  "experiment_id": 42,
  "budget_allocated_aud": 72.92
}
```

Response 404: Experiment not found.

Response 400: Experiment not in 'pending' status.

```
curl -X POST http://localhost:5000/approve/42 \
  -H "Authorization: Bearer $GATE_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"approved_by": "martin"}'
```

POST /reject/

Reject a pending or approved experiment.

Request body (optional):

```
{
  "reason": "Budget too tight this week",
  "rejected_by": "martin"
}
```

Response 200:

```
{
  "status": "rejected",
  "experiment_id": 42,
  "reason": "Budget too tight this week"
}
```

```
curl -X POST http://localhost:5000/reject/42 \
  -H "Authorization: Bearer $GATE_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"reason": "Not aligned with current strategy", "rejected_by": "martin"}'
```

GET /dashboard

Full swarm status overview. Returns swarm summary, portfolio breakdown, pending/active experiments, recent leads, and circuit breaker history.

Response:

```
{
  "swarm": {
    "active_experiments": 3,
    "pending_approval": 1,
    "budget_used_aud": 120.50,
    "circuit_breaker": "green"
  },
  "portfolio": { ... },
  "pending_experiments": [...],
  "active_experiments": [...],
  "recent_leads": [...],
  "circuit_breaker_history": [...],
  "generated_at": "2026-03-09T08:00:00"
}
```

```
curl -H "Authorization: Bearer $GATE_API_KEY" http://localhost:5000/dashboard
```

GET /experiments

List recent experiments with optional status filter.

Query params:

PARAM	DEFAULT	DESCRIPTION
`status`	(none)	Filter by status: pending/approved/running/complete/killed/rejected
`limit`	20	Max results (capped at 100)

Response:

```
{
  "experiments": [...],
  "count": 5
}
```

```
# All recent
curl -H "Authorization: Bearer $GATE_API_KEY" http://localhost:5000/experiments

# Running only
curl -H "Authorization: Bearer $GATE_API_KEY" \
  "http://localhost:5000/experiments?status=running&limit=10"
```

GET /costs

API usage and cost breakdown.

Query params:

PARAM	DEFAULT	DESCRIPTION
`days`	7	Look-back period (max 365)
`call_id`	(none)	Cost for a single voice call
`client_id`	(none)	Cost for a specific client

Response (no filters):

```
{
  "summary": {
    "period_days": 7,
    "breakdown": {
      "openai": {"cost_usd": 1.24, "calls": 48},
      "retell": {"cost_usd": 0.84, "calls": 12}
    },
    "total_usd": 2.08,
    "total_aud": 3.22
  },
  "daily": [...],
  "projection": {
    "last_7_days_usd": 2.08,
    "projected_month_usd": 8.91,
    "projected_month_aud": 13.82,
    "margin_note": "At $14 AUD/month costs vs $1,500-2,000 retainer = ~99% margin"
  }
}
```

```
# Full summary (7 days)
curl -H "Authorization: Bearer $GATE_API_KEY" http://localhost:5000/costs

# 30-day view
curl -H "Authorization: Bearer $GATE_API_KEY" "http://localhost:5000/costs?days=30"

# Single call
curl -H "Authorization: Bearer $GATE_API_KEY" \
  "http://localhost:5000/costs?call_id=call_abc123"

# Per client
curl -H "Authorization: Bearer $GATE_API_KEY" \
  "http://localhost:5000/costs?client_id=sunpower_perth&days=30"
```

POST /approve-breaker

Reset a Red circuit breaker. Requires human approval.

Request body:

```
{
  "approved_by": "martin"
}
```

Response 200:

```
{
  "success": true,
  "message": "Circuit breaker reset by martin"
}
```

```
curl -X POST http://localhost:5000/approve-breaker \
  -H "Authorization: Bearer $GATE_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"approved_by": "martin"}'
```

POST /slack/actions

Handles Slack interactive button callbacks (approve/reject experiment buttons posted by the swarm). Validates the request using `SLACK_SIGNING_SECRET` + timestamp. Requests older than 5 minutes are rejected (replay protection).

Request: Form-encoded `payload` field containing JSON (sent automatically by Slack).

Response 200:

```
{
  "status": "ok",
  "result": {"status": "approved", "experiment_id": 42, "budget_allocated_aud": 72.92}
}
```

Response 403: Invalid Slack signature.

This endpoint is called by Slack automatically — not for direct use.

GHL Webhook Server — Port 5001

Receives events from GoHighLevel. All endpoints verify `X-GHL-Signature` HMAC-SHA256 header using `GHL_WEBHOOK_SECRET`. If the secret is not configured, the check is skipped with a warning.

GET /health

```
curl http://localhost:5001/health
```

Response:

```
{
  "status": "ok",
  "service": "ghl-webhooks",
  "timestamp": "2026-03-09T08:00:00"
}
```

POST /webhook/new-lead

New contact created in GHL. Saves to the `leads` table and triggers qualification scoring. If score >= 7 and phone is present, triggers an outbound Retell call.

GHL webhook event: Contact Created

Payload fields read:

FIELD	GHL KEY
name	`full_name`, `first_name` + `last_name`, or `name`
phone	`phone` or `phoneRaw`
email	`email`
suburb	`suburb` or `city`
state	`state`
homeowner_status	`homeowner_status` or `customField.homeowner_status`
monthly_bill	`monthly_bill` or `customField.monthly_bill`
roof_type	`roof_type` or `customField.roof_type`
roof_age	`roof_age` or `customField.roof_age`
pipeline_stage	`pipelineStage` or `pipeline_stage`

Response 200:

```
{
  "status": "processed",
  "lead_id": 7,
  "score": 8.2
}
```

```
# Test with a simulated lead
curl -X POST http://localhost:5001/webhook/new-lead \
  -H "Content-Type: application/json" \
  -d '{
    "full_name": "Jane Smith",
    "phone": "+61412345678",
    "email": "jane@example.com",
    "suburb": "Subiaco",
    "state": "WA",
    "homeowner_status": "owner",
    "monthly_bill": 350,
    "roof_type": "tile",
    "roof_age": 5
  }'
```

POST /webhook/call-complete

Voice AI call finished event from GHL. Updates the lead record with call outcome and timestamps.

GHL webhook event: Conversation Status Changed

Payload fields read: `contactId`, `outcome`, `phone`

Response 200:

```
{
  "status": "processed",
  "contact_id": "abc123",
  "outcome": "booked_assessment"
}
```

POST /webhook/form-submit

Solar quote form submitted via GHL Funnel. Processes richer form data including electricity bill and homeowner status. Saves and qualifies the lead.

Payload fields read: Same as new-lead, plus `homeowner` (maps to homeowner_status), `electricity_bill` or `bill` (maps to monthly_bill), `roof` (maps to roof_type).

Response 200:

```
{
  "status": "processed",
  "lead_id": 8,
  "score": 6.5
}
```

POST /webhook/stage-change

Pipeline stage changed in GHL. Updates `pipeline_stage` on the matching lead record. If the new stage contains "convert", also sets `status = 'converted'`.

GHL webhook event: Opportunity Stage Change

Payload fields read: `contactId` (or `contact_id`), `newStage` (or `stage`)

Response 200:

```
{
  "status": "processed",
  "contact_id": "abc123",
  "new_stage": "Proposal Sent"
}
```

Voice Webhook Server — Port 5002

Receives events from Retell AI. In-memory call context stored in `_call_contexts` dict keyed by `call_id`.

GET /voice/health

```
curl http://localhost:5002/voice/health
```

Response:

```
{"status": "ok", "service": "voice-webhook"}
```

POST /voice/call-started

Retell sends this when a call is initiated. Initialises call context with `client_id`, phones, and empty `lead_data`.

Payload:

```
{
  "call_id": "call_abc123",
  "from_number": "+61412345678",
  "to_number": "+61892345678",
  "agent_id": "agent_xyz"
}
```

Response 200:

```
{"status": "ok"}
```

POST /voice/response

Main LLM endpoint. Retell sends the transcript, the server calls GPT-4o with the full system prompt, and returns the next response. This endpoint is called on every agent turn during a call.

Payload (Retell Custom LLM format):

```
{
  "call_id": "call_abc123",
  "interaction_type": "response_required",
  "response_id": 5,
  "transcript": [
    {"role": "agent", "content": "Hi! Thanks for calling SunPower Perth."},
    {"role": "user", "content": "Hi, I'm interested in solar panels."}
  ]
}
```

Response:

```
{
  "response_id": 5,
  "content": "Great! Let me ask you a few quick questions to see if solar is right for your home.
  "content_complete": true,
  "end_call": false
}
```

POST /voice/elevenlabs/response

Alternative LLM endpoint used when ElevenLabs Conversational AI is the telephony layer instead of Retell. Accepts the same format as `/voice/response`.

POST /voice/post-call

Retell sends the full call data after hang-up. Triggers 9-step post-call processing: extract, merge, upsert lead, score, update GHL, create task, update call_logs, Slack notification, cold ledger.

Payload:

```
{
  "call_id": "call_abc123",
  "transcript": [...],
  "duration_ms": 180000,
  "recording_url": "https://retell.com/recordings/...",
  "agent_id": "agent_xyz"
}
```

Response 200:

```
{"status": "processed", "call_id": "call_abc123"}
```

Dashboard API — Port 5003

CORS-enabled. Allows requests from `http://localhost:5173` (Vite dev), `http://localhost:4173` (Vite preview), and `FRONTEND_URL` env var. The React swarm board polls these endpoints every 30 seconds.

No authentication by default (internal network service). Add `FRONTEND_URL` in `.env` for production deployments.

GET /api/health

Service health with CRM connection status.

Response:

```
{
  "status": "ok",
  "service": "dashboard-api",
  "crm": {
    "active": "ghl",
    "ghl": true,
    "hubspot": false,
    "salesforce": false
  },
  "timestamp": "2026-03-09T08:00:00"
}
```

```
curl http://localhost:5003/api/health
```

GET /api/crm/status

Which CRM is active and all configured integrations.

Response:

```
{
  "active": "ghl",
  "configured": ["ghl"],
  "detail": {
    "active": "ghl",
    "ghl": true,
    "hubspot": false,
    "salesforce": false
  }
}
```

```
curl http://localhost:5003/api/crm/status
```

GET /api/crm/pipeline

Pipeline stages with contact counts. Served from `crm_cache` table (refreshed every 30 minutes by `crm_sync`). Falls back to a live CRM call if the cache is empty.

Response:

```
{
  "source": "cache",
  "stages": [
    {"name": "New Lead", "count": 12},
    {"name": "Qualified", "count": 5},
    {"name": "Proposal Sent", "count": 3}
  ],
  "cached_at": "2026-03-09T07:30:00"
}
```

```
curl http://localhost:5003/api/crm/pipeline
```

GET /api/crm/contacts

Recent contacts from the cache table.

Query params:

PARAM	DEFAULT	DESCRIPTION
`limit`	20	Max results

Response:

```
{
  "contacts": [...],
  "count": 20
}
```

```
curl "http://localhost:5003/api/crm/contacts?limit=10"
```

GET /api/crm/metrics

Conversion funnel metrics from `crm_cache` (`metrics_summary` key).

Response:

```
{
  "source": "cache",
  "metrics": {
    "total_contacts": 147,
    "new_this_week": 23,
    "pipeline_stages": [...],
    "conversion_rate": 0.12,
    "synced_at": "2026-03-09T07:30:00"
  }
}
```

```
curl http://localhost:5003/api/crm/metrics
```

GET /api/swarm/summary

Hot memory swarm overview: active experiments, pending approvals, budget used, circuit breaker state.

Response:

```
{
  "active_experiments": 3,
  "pending_approval": 1,
  "budget_used_aud": 120.50,
  "circuit_breaker": "green"
}
```

```
curl http://localhost:5003/api/swarm/summary
```

GET /api/swarm/experiments

Recent experiments, optionally filtered by status.

Query params:

PARAM	DEFAULT	DESCRIPTION
`status`	(none)	Filter: pending/approved/running/complete/killed/rejected
`limit`	15	Max results

```
curl "http://localhost:5003/api/swarm/experiments?status=running"
```

GET /api/swarm/leads

Recent leads from the `leads` table. Returns id, name, qualification_score, recommended_action, status, created_at.

Query params:

PARAM	DEFAULT	DESCRIPTION
`limit`	20	Max results

```
curl "http://localhost:5003/api/swarm/leads?limit=50"
```

GET /api/swarm/circuit-breaker

Current circuit breaker state.

Response:

```
{
  "level": "green",
  "halted": false
}
```

```
curl http://localhost:5003/api/swarm/circuit-breaker
```

GET /api/board/state

`board-state.json` merged with live DB counts. Used by the React board's Overview tab.

Response:

```
{
  "...board-state.json contents...",
  "liveStats": {
    "experiments": {
      "pending": 1,
      "approved": 2,
      "running": 1,
      "complete": 8
    },
    "totalLeads": 147,
    "abTestsRunning": 2,
    "generatedAt": "2026-03-09T08:00:00"
  }
}
```

```
curl http://localhost:5003/api/board/state
```

Auth & Admin Blueprints (Port 5003)

Additional routes registered as Flask blueprints on the Dashboard API:

BLUEPRINT	PREFIX	PURPOSE
`api/auth.py`	`/auth/...`	Login, JWT token issue/refresh
`api/users_api.py`	`/users/...`	User management (owner/admin/client roles)
`api/settings_api.py`	`/settings/...`	Runtime config overrides (`app_settings` table)
`api/company_api.py`	`/companies/...`	Solar SME client profiles
`api/apikeys_api.py`	`/apikeys/...`	Embed/webhook API key management

These blueprints use JWT auth (`PyJWT`) and bcrypt password hashing. The `seed_owner()` call on startup creates the initial owner account if none exists.